

Clan

A Nix framework for distributed services

Romain Paquet (rpqt)

11/09/2026

GreNUG Meetup #1

- What's a distributed service? 2
- Clan 7
- Inventory 10
- Vars 16
- Networking 21
- Exports 24

What's a distributed service?

What's a distributed service?

For the purpose of this talk, a distributed service is an application involving a **network of interconnected machines known at deployment time**.

Examples:

- Mesh VPNs (Tailscale, Wireguard, Zerotier, ...)
- P2P applications (Syncthing, Radicle, ...)
- CI/CD orchestration systems
- SSH

A standalone web server doesn't qualify:

- the clients are only known at run-time
- connections are temporary

Distributed services are difficult

Configuring these services can be challenging :

- both **individual** (per-machine) and **shared** (per-service) configuration.
- per-deployment parameters (public/private keys, IP addresses)
- add a machine in a P2P service → re-configure all the others

NixOS helps but it is still a lot of work.

Homework: look at the [NixOS wiki Wireguard page](#)

Outsourcing as an escape?

Distributed services are difficult, but you can let others to manage it for you :

- Wireguard → Tailscale
- Syncthing → Cloud storage
- Internal DNS & TLS → Rent a domain from a registrar
- VPN → no need for that, just rent a few IPs
- SaaS

But most of the time this is **not declarative/reproducible** 🤬

→ for this you need control over your infra

Making it easier

Clan is all about sovereign infrastructure.

Clan tries to make deploying these distributed services **way easier** (as well as helping with a lot of the infra management problems).

It does so by creating abstractions for re-usable distributed services.

This allows for smoother definition and composition of such services, but also moves a lot of the complexity **from the user to the maintainers**.

Clan



The Clan

A **Clan** is a network of machines running a set of (distributed) services.

You can create a Clan interactively using the `clan` CLI. It will prompt you for some informations and set up a Nix flake. By default, Clan-related options are set in a `clan.nix` file. Let's look at what you can put in this file.

💡 *Clan exposes a `flake-parts` module under the hood, so you can also import it into an existing Flake and test/migrate gradually.*

Clan meta

A Clan has a **name** and a **domain**. The domain is a TLD or subdomain that can be used for Clan-internal DNS, e.g. `.clan`, `.aura` or whatever...

```
{  
  clan.meta.name = "grenoble-clan";  
  clan.meta.domain = "gre";  
}
```

You can use it to connect to other machines/services in the Clan with e.g. `laptop.gre` / `immich.gre` with TLS etc (optional).

Inventory

The Inventory

The Clan **Inventory** is the user-facing interface where you declare :

- the set of **machines** inside the Clan (+ metadata)
- the set of **services** and how they involve these machines

It can be defined from Nix and in an `inventory.json` file (merged together).

Interfacing through JSON allows to build additional front-ends such as GUIs. Clan aims to democratize computing, and not everyone wants to write Nix code...

Inventory machines

```
{
  clan.inventory.machines = {
    alice-laptop = { tags = ["laptop"]; name = "alice"; };
    bob-laptop   = { tags = ["laptop"]; name = "bob";   };
    backup-server = { tags = ["server"]; name = "backup-01"; };
  }
}
```

Tags allow to refer to groups of machines, both from the inventory and the CLI
e.g. `clan machines update --tags server` to re-deploy all servers.

Actual machine configuration is auto-included from
`machines/{hostname}/configuration.nix`. The Clan CLI can help you add new machines by generating the base config.

Inventory services

The Inventory also contains service “**instances**”. Multiple instances of the same service can co-exist.

```
{
  clan.inventory.instances = {
    # name of the instance
    home-wifi = {
      module.name = "wifi"; # which service to instantiate

      # Enables the service on all machines tagged "laptop"
      roles.default.tags = [ "laptop" ];
    };
  };
}
```

Example: borgbackup service

Two **roles** can be attributed to machines:

- **client**: machines that are backed up
- **server**: machines where backups are stored

```
{  
    inventory.instances.borgbackup = {  
        roles.client.machines."alice-laptop" = { };  
  
        roles.server.machines."backup-server" = {  
            settings.address = "backup-server.clan";  
            settings.directory = "/var/lib/borgbackup";  
        };  
    };  
}
```

A large catalog of services

Many services are available upstream:

- **Core services:** 3+ DNS variants, TLS, SSH, P2P SSH, ...
- **Apps:** Garage, Syncthing, BorgBackup, Authelia...

They are all tested in CI using the NixOS VM test framework (QEMU or containers), in multi-machines scenarios.

Writing your own service is not too difficult.

Vars



Problem

Many services need pre-generated secrets

- Public/private keys, passwords, tokens
- They cannot be stored as plaintext in the world-readable Nix store ⚠

The typical recommended solution is to use `agenix` or `sops-nix`

- You declare secrets and target public keys with NixOS options
- You provide secret values with in-repo encrypted files (CLI/helpers)
- You plug it to service options (systemd-credentials / env vars / files)

A new solution: Clan Vars

Clan has its own secret management system called **Vars**

- The Clan service declares the required secrets and how to generate them
- **Most secrets end up auto-generated**
- The user is only prompted for things like passwords and external API tokens

Clan Vars features

- Multiple backends (**sops** or **age** for now)
- Encrypted secrets are stored in the repo
 - VCS-tracked
 - Can be public
 - Private keys live wherever you want, e.g. Yubikey/HSM
- Also supports public values directly accessible from Nix, e.g. IP addresses
- Secrets can be derived from other secrets
 - Changing a dependency recomputes the derived values
 - e.g. PKI/certificates
 - e.g. generate an IPv6 address from a private key

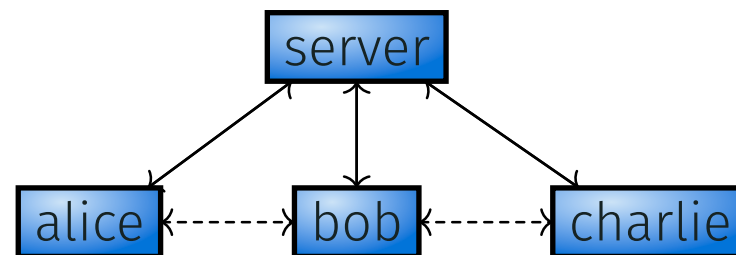
Vars example: SSH key

```
{
  clan.core.vars.generators."sshd-root-key" = {
    files."id_ed25519" = { };
    files."id_ed25519.pub".secret = false;
    runtimeInputs = [ pkgs.openssh ];
    script = ''
      ssh-keygen -t ed25519 -N "" -f "$out"/id_ed25519 \
        -C "root@${config.clan.core.settings.machine.name}"
    '';
  }
}
```

Networking

Wireguard

```
{  
  inventory.instances."wg0" = {  
    module.name = "wireguard";  
    roles.controller.machines."server" = {  
      settings.endpoint = "vpn.example.com";  
    };  
    roles.peer.tags = ["all"];  
  };  
}
```



This is all we need! IPv6 addresses and keys are generated with Vars.

Clan is starting to have a unified networking layer, currently using the Yggdrasil service along with multiple mesh VPNs:

- Provides a **single clan-internal IPv6 for each machine**
- **Routes traffic** through the fastest underlying networks (Internet, LAN, Wireguard, Zerotier, Tailscale, Mycelium, ...)

Exports

Exports are one of the killer features of Clan, however there are not completely documented/stable yet.

What they allow is dependancy between services: each service can expose information about what it provides

- the multi-VPN setup mentionned earlier (IPs export)
- **auto-setup PKI/CA/TLS** for web sercices (endpoints export)
- **auto-setup OAuth** applications (auth export)
- replace a backup service with another backend

Other goodies born from Clan

data-mesher

P2P sync for runtime state, *e.g. distribute Clan-internal DNS zones and update them in real-time*

A pure Nix query language

```
clan select 'nixosConfigurations.*.config.networking.interfaces.*.{name}'
```

nixos-facter

Replacement for `hardware-configuration.nix`: facter scans the machine before installation to produce a large detailed JSON report about the hardware, which is then consumed from Nix.